

Numele si prenumele (cu MAJUSCULE): _____ Grupa: _____

Test: _____ Tema: _____ Colocviu: _____ FINAL: _____

Test de laborator - Arhitectura Sistemelor de Calcul

ianuarie 2025
Seria 15

- Nota maxima pe care o puteti obtine este 10.
- Nota obtinuta trebuie sa fie minim 5 pentru a promova, fara nicio rotunjire superioara.
- Orice tentativa de fraudă este considerata o incalcare a Regulamentului de Etica!

1 Partea 0x00: x86 - maxim 6p

Presupunem ca aveti acces la un executabil `exec`, pe care il inspectati cu `objdump -d exec`. In momentul in care rulati aceasta comanda, va opriti asupra urmatorului cod. Analizati-l si raspundeti intrebarilor de mai jos. Pentru fiecare raspuns in parte, veti preciza si instructiunile care v-au ajutat in rezolvare.

```
08049176 <f>:
17a: 55          push    %ebp
17b: 89 e5       mov     %esp,%ebp
17d: 83 ec 10    sub     $0x10,%esp
185: 05 7b 2e 00 00 add     $0x2e7b,%eax
18a: c7 45 fc 01 00 00 00 movl    $0x1,-0x4(%ebp)
191: c7 45 f8 00 00 00 00 movl    $0x0,-0x8(%ebp)
198: eb 40       jmp     1da <f+0x64>
19a: 8b 45 f8    mov     -0x8(%ebp),%eax
19d: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
1a4: 8b 45 08    mov     0x8(%ebp),%eax
1a7: 01 d0       add     %edx,%eax
1a9: 8b 00       mov     (%eax),%eax
1ab: 0f af 45 10 imul    0x10(%ebp),%eax
1af: 89 c2       mov     %eax,%edx
1b1: 8b 45 14    mov     0x14(%ebp),%eax
1b4: 0f af 45 18 imul    0x18(%ebp),%eax
1b8: 39 c2       cmp     %eax,%edx
1ba: 7e 1a       jle     1d6 <f+0x60>
1bc: 8b 45 f8    mov     -0x8(%ebp),%eax
1bf: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
1c6: 8b 45 08    mov     0x8(%ebp),%eax
1c9: 01 d0       add     %edx,%eax
1cb: 8b 00       mov     (%eax),%eax
1cd: 8b 55 fc    mov     -0x4(%ebp),%edx
1d0: 0f af c2    imul    %edx,%eax
1d3: 89 45 fc    mov     %eax,-0x4(%ebp)
1d6: 83 45 f8 01 addl    $0x1,-0x8(%ebp)
1da: 8b 45 f8    mov     -0x8(%ebp),%eax
1dd: 3b 45 0c    cmp     0xc(%ebp),%eax
1e0: 7c b8       jl      19a <f+0x24>
1e2: 8b 45 fc    mov     -0x4(%ebp),%eax
1e6: c3         ret

g00: <g>:
g01:          pushl   %ebp
g02:          movl    %esp, %ebp
g03:          subl    $16, %esp
g04:          movl    8(%ebp), %eax
g05:          movss   (%eax), %xmm0
g06:          cvttss2sil %xmm0, %eax
g07:          pushl   %eax
g08:          pushl   $5
g09:          pushl   $5
g0A:          pushl   12(%ebp)
g0B:          pushl   8(%ebp)
g0C:          call    f
g0D:          addl    $20, %esp
g0E:          movl    %eax, -12(%ebp)
g0F:          fldz
g10:          fstps   -8(%ebp)
g11:          movl    $0, -4(%ebp)
g12:          jmp     .L9
g13: .L10:
g14:          flds    -8(%ebp)
g15:          movl    -4(%ebp), %eax
g16:          leal    0(,%eax,4), %edx
g17:          movl    8(%ebp), %eax
g18:          addl    %edx, %eax
g19:          flds    (%eax)
g1A:          fldl    .LC2
g1B:          fmulp   %st, %st(1)
g1C:          faddp   %st, %st(1)
g1D:          fstps   -8(%ebp)
g1E:          addl    $1, -4(%ebp)
g1F: .L9:
g20:          movl    -4(%ebp), %eax
g21:          cmpl    12(%ebp), %eax
g22:          jl      .L10
g23:          flds    -8(%ebp)
g24:          ret
```

a. (0.75p) Cate argumente primeste procedura `f` si cum ati identificat acest numar de argumente?

b. (0.75p) Ce tip de date returneaza procedura `f` si cum ati identificat acest tip?

- c. (0.75p) In timp ce analizati executabilul, observati in sectiunea `.data` valoarea `0x40B00000`. In timp ce cititi codul, va dati seama ca aceasta valoare este, de fapt, o reprezentare pe formatul `single` a unei valori rationale. Despre ce valoare este vorba?
- d. (0.75p) Va atrage atentia codificarea hexa a instructiunilor, si doriti sa vedeti care este semantica acestora. Specific, analizati instructiunile `mov` si observati ca ele difera destul de mult ca reprezentare, incat chiar si `opcode`-ul difera intre doua instructiuni `mov`. Stiind ca aceste `opcode`-uri sunt `89`, `c7` si `8b`, identificati cand se foloseste fiecare. In cazul codificarii `c7` analizati ultimii 4 Bytes din fiecare reprezentare - ce semnifica acea valoare?
- e. (0.5p) Procedura `f` contine o structura repetitiva. Identificati toate elementele acestei structuri: initializarea contorului, conditia de a ramane in structura, respectiv pasul de continuare (operatia asupra contorului).
- f. (1p) Analizati acum procedura `g`. Primul lucru pe care il observati sunt instructiunile specifice pentru a lucra cu `floating point`. Identificati `fldz` care incarca 0 peste stiva FPU (in `%st(0)`), `fildl op` care incarca intregul `op` ca `float` pe stiva FPU (in `%st(0)`), `fmulp op1, op2` care efectueaza pe formatul `float` operatia `op2 := op2 * op1` si apoi efectueaza `pop`, `faddp op1, op2` care efectueaza pe formatul `float` operatia `op2 := op2 + op1` (in cazul `faddp %st, %st(1)` efectueaza `%st(1) += %st(0)` si efectueaza `pop`). Avand aceste informatii, determinati care este structura repetitiva si ce se calculeaza in acea structura.

- g. (0.5p) Considerati rescrierea instructiunilor pe stiva FPU din procedura **g** in SIMD. Care este echivalentul lor? (pentru liniile g14-g1E).
- h. (1p) Observati ca la linia g0C, din procedura **g** se face un *call* imbricat in procedura **f**. Reprezentati configuratia stivei de apel, in momentul in care se obtine adancimea maxima, considerand reprezentarea incepand cu argumentele primite de **g**.

2 Partea 0x01: RISC-V - maxim 3p

- a. (0.75p) Au toate instructiunile RISC-V o varianta comprimata? Daca da, explicati de ce. Daca nu, prezentati 2 situatii diferite in care instructiunile nu sunt comprimabile si motivati. De ce x86 nu are o astfel de extensie?
- b. (0.75p) Se considera un procesor RISC-V minimal (RV32I) care nu implementeaza niciun mecanism de securitate. Care este efectul codului de mai jos daca se ruleaza pe un astfel de procesor? Descrieti fiecare pas de la inceputul pana la finalul executiei (Mentionati la fiecare pas cum se modifica registrii).

```
.data
x: .long 0x00ee0333

.text
.global main
main:
la a0, x
jr a0

li a7, 93
li a0, 0
ecall
```


- c. (0.75p) Ce valoare va fi depozitata in **a0** in urma executiei urmatoarelor instructiuni, stiind ca **pc** este intial 0? Prezantati efectul fiecarei instructiuni.

```
auipc a0, 0x12345
slli a0, a0, 4
auipc a1, 0x1
add a0, a0, a1
```

- d. (0.75p) Se da urmatorul schelet de functie. Care este efectul sau?

```
proc:
addi sp, sp, -8
sw ra, 4(sp)
sw s0, 0(sp)
addi sp, sp, -8

// Cod care nu mai modifica valoarea lui sp

lw s0, 0(sp)
lw ra, 4(sp)
addi sp, sp, 16
ret
```

3 Partea 0x02: Performanta si cache - maxim 1p

- a. (0.5p) Un procesor are un pipeline cu 5 stadii (Fetch, Decode, Execute, Memory, Write-back). Se considera urmatoarea secventa de instructiuni:

```
lw a0, 0(gp)
add a1, a0, a2
sub a3, a1, a0
```

Identificati hazardurile de date si tipul lor. Presupunem ca folosim mecanismul de forwarding pentru a le rezolva. Astfel pentru ca o instructiune dintr-un hazard sa poata intra in stadiul de execute, instructiunea dependinta trebuie sa finalize stadiul de memory daca este o instructiune de accesare a memoriei, respectiv de execute daca este un alt tip de instructiune. In acest context, care este numarul total de cicluri in care se termina de executat intreaga secventa? (Pentru usurinta puteti realiza un tabel cu ciclurile.)

- b. (0.5p) Un sistem are o memorie principală de 2^{18} bytes iar cache-ul are o capacitate totală de 8 KB, cu o dimensiune a unui bloc de 128 bytes (atât pentru memoria principală, cât și pentru cache). Calculați numărul total de blocuri din memoria principală. Determinați numărul de linii (blocuri) din cache. În cazul unei scheme de mapare directă, unde va fi mapată adresa 0xA1F0 în cache?